

Stack Data Structure Operations

Technical Reference & Complexity Guide

A **Stack** is a fundamental linear data structure that operates under the **LIFO (Last In, First Out)** organizational principle. In this architecture, elements are stacked sequentially, and all insertion and extraction operations are strictly restricted to the topmost position of the structure.

1. Core Operations (Mutations)

These primary functions directly alter the structural state, elements, and memory boundary allocation of the stack:

- **Push:** Inserts a brand new element onto the top of the stack. If the execution environment attempts a push onto a static stack that has reached its maximum defined boundary capacity, it triggers a *Stack Overflow* failure state.
- **Pop:** Removes and returns the current top element from the stack. If a pop operation is requested on a stack that contains zero elements, it triggers a *Stack Underflow* failure state.

2. Inspection & Query Operations

These auxiliary procedures read or assert properties of the stack configuration without executing any structural adjustments or state transitions:

- **Peek / Top:** Returns the data contents of the element currently sitting at the top position without removing it from the sequence. This serves as a vital verification step before consumption.
- **IsEmpty:** Evaluates the internal pointers or size tracking counter to return a boolean truth value indicating whether the stack contains zero elements.
- **IsFull:** Relevant primarily to array-based static memory stack definitions; evaluates whether the active tracking boundaries match the maximum allocated physical memory limits.
- **Size / Count:** Reports an integer tracking the current total quantity of elements allocated within the stack sequence.

3. Structural Lifecycle Operations

- **Initialize / Create:** Builds the initial underlying physical allocation context, initializing the index control bounds (typically setting a top array pointer to -1 or null reference).
- **Clear / Reset:** Iterates or updates control reference boundaries to immediately drop all stored contents, recycling memory space and returning the stack instantly back to a valid empty state.

4. Algorithm Efficiency & Time Complexity

Stacks provide deterministic execution properties because they isolate modifications exclusively to a single edge. The structural complexities map consistently as follows:

Operation Name	Runtime Complexity	Auxiliary Space Complexity
Push	$O(1)$	$O(1)$
Pop	$O(1)$	$O(1)$
Peek / Top	$O(1)$	$O(1)$
IsEmpty	$O(1)$	$O(1)$
Size / Count	$O(1)$	$O(1)$

Error Prevention Rule: Production systems must always execute validation logic using `IsEmpty` prior to calling `Pop`, and `IsFull` prior to executing `Push`, ensuring robust failure avoidance against boundary exceptions.